

Inteligência Computacional

Aula 14: Algoritmos Genéticos

Prof. Eduardo Corrêa Gonçalves

09/10/2026

Sumário

Introdução

O que é Algoritmo Genético (AG) ?

Resumo do AG básico.

AGs – conhecendo através de um exemplo prático

Problema de Roteamento de Veículos (PRV)

Introdução aos AGs

AGs em ação !

Outras aplicações:

- (i) otimizando o k-Médias;
- (ii) otimizando prompts do ChatGPT;
- (iii) seleção de atributos de classificadores.

Introdução (1/4)

- O que é Algoritmo Genético (AG) ?
 - Método de **busca e otimização** que trabalha baseado na aplicação de **princípios Darwinianos** para resolver problemas difíceis:
 - Evolução e Seleção Natural
 - Reprodução
 - Mutação
 - O método resolve problemas mantendo uma **população de soluções candidatas**.
 - A população é **aprimorada através de gerações sucessivas**, utilizando **seleção**, **reprodução** (*crossover*) e **mutação**.



Introdução (2/4)

- AG's representam um **método para resolver problemas**.
 - Um tipo de *framework*.

Outline of the Basic Genetic Algorithm

1. **[Start]** Generate a random population of n chromosomes (suitable solutions for the problem)
2. **[Fitness]** Evaluate the fitness $f(x)$ of each chromosome x in the population
3. **[New population]** Create a new population by repeating the following steps until the new population is complete
 1. **[Selection]** Select two parent chromosomes from the population according to their fitness (the better the fitness, the greater the chance of being selected)
 2. **[Crossover]** With a crossover probability cross over the parents to form a new offspring (children). If no crossover is performed, the offspring is an exact copy of the parents.
 3. **[Mutation]** With a mutation probability, mutate the new offspring at each locus (position in the chromosome).
 4. **[Accepting]** Place the new offspring in the new population
4. **[Replace]** Use the newly generated population for the next run of the algorithm
5. **[Test]** If the end condition is satisfied, **stop** and return the best solution in the current population
6. **[Loop]** Go to step 2

Introdução (3/4)

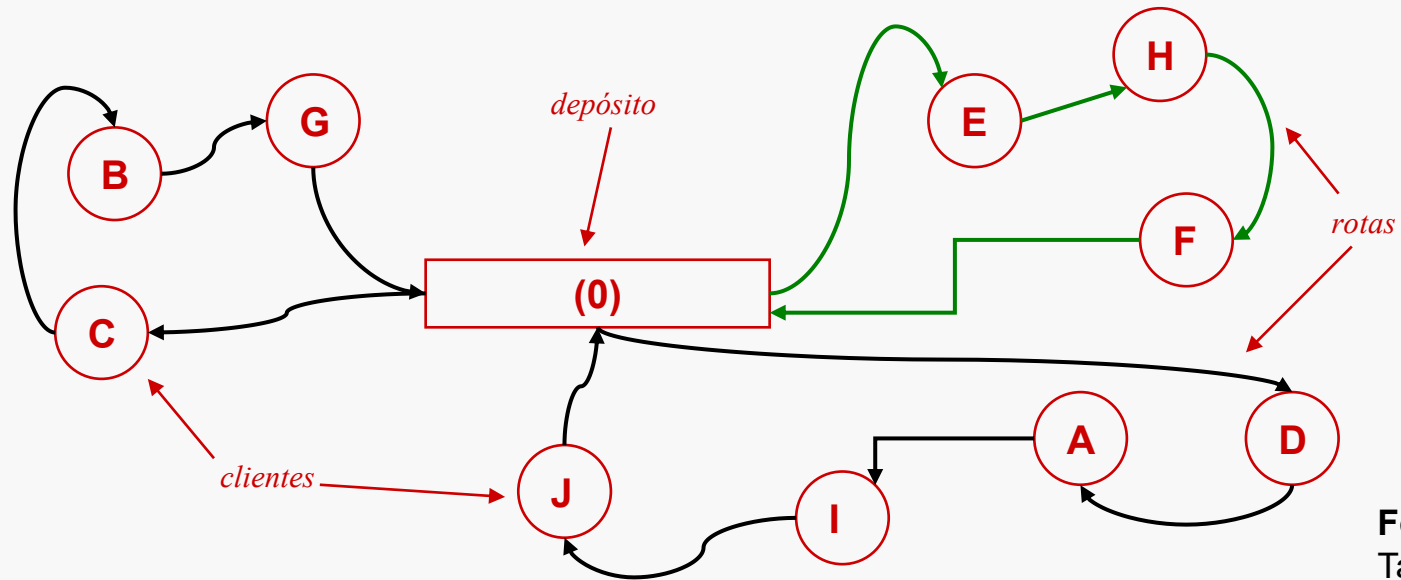
- **AG é um Método Estocástico**
 - Um AG se desenvolve ao longo do tempo de uma maneira (parcialmente) aleatória e imprevisível.
- **AG é um Método de CIN**
 - AGs são uma das mais conhecidas e utilizadas técnicas de **Computação Inspirada pela Natureza** (CIN)
 - Estes algoritmos são utilizados especialmente na área de Inteligência Computacional
 - Outros exemplos de algoritmos de CIN são:
 - Redes Neurais
 - *Particle Swarm Optimization* (Otimização por Enxame de Partículas)
 - *Ant Colony Optimization* (Otimização por Colônia de Formigas)
 - Detalhes em: <https://www.fabriciobreve.com/material/cin/>

Introdução (4/4)

- **AG é um Método de Busca (*search method*)**
 - Quando resolvemos um problema normalmente **buscamos pela melhor solução** em um conjunto soluções candidatas.
 - O **conjunto com todas as soluções possíveis** é chamado de **espaço de busca**.
 - AGs são utilizados especialmente para resolver problemas com um número “**combinatorialmente explosivo**” de **soluções candidatas** (por exemplo, **exponencial** ou **fatorial**).
 - Nos próximos slides, apresentaremos o Problema do Roteamento de Veículos, que possui a característica acima.
 - Veremos que o AG é usado para navegar pelo espaço de busca do PRV de uma forma inteligente em busca uma “boa solução”.

Problema do Roteamento de Veículos - PRV (1/10)

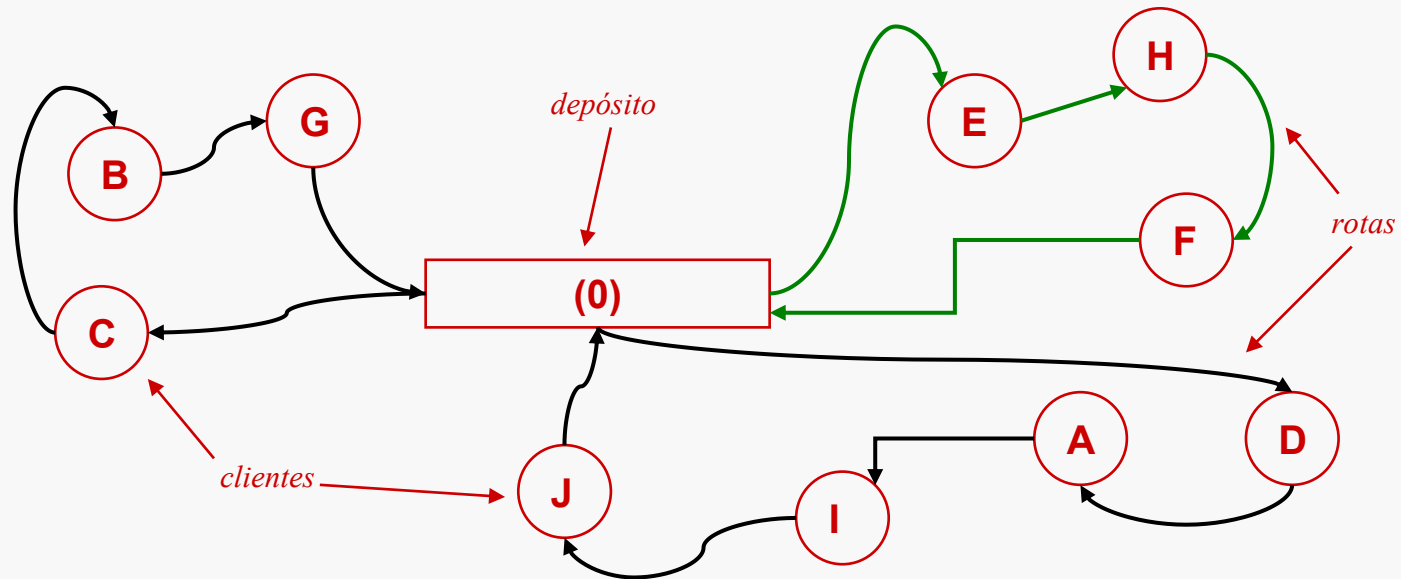
- Definição do PRV (versão básica)
 - Existe um **depósito central** (0), que utiliza k **veículos** com **capacidade** idêntica (C) para servir **demandas** (d_i) para n **clientes**, $i=1 \dots n$.



Fonte:
Tavares et al.,
2002

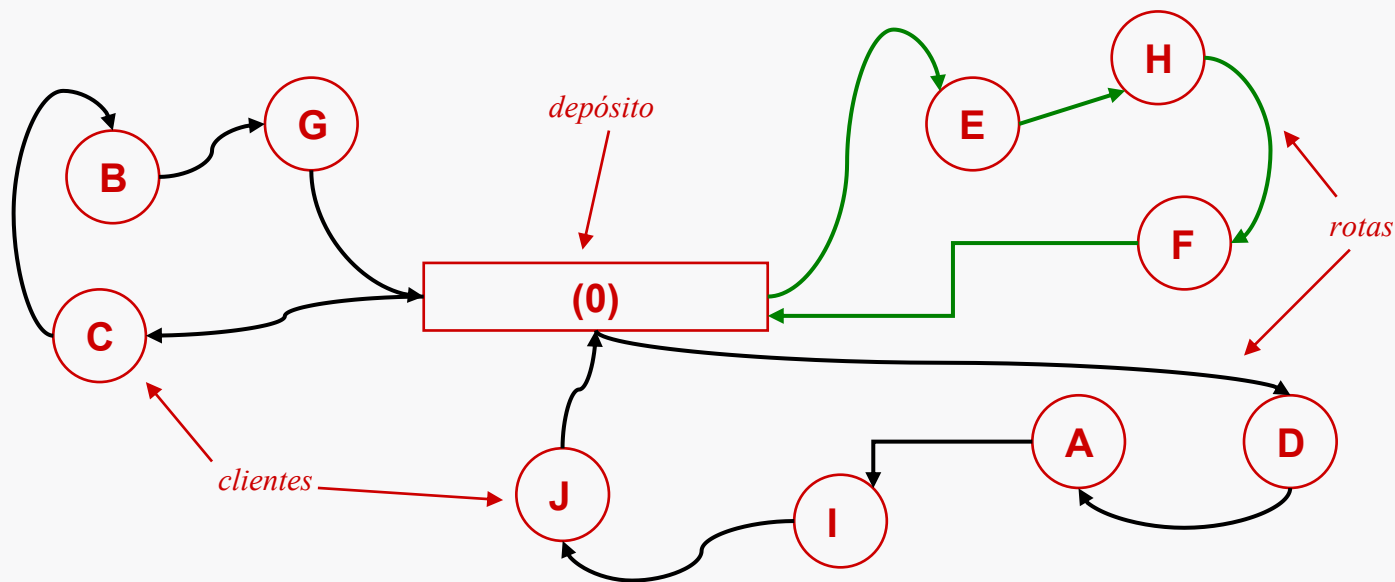
Problema do Roteamento de Veículos - PRV (2/10)

- Definição do PRV (versão básica)
 - Os veículos devem realizar as entregas com um **custo total mínimo**. O custo total representa o somatório das distâncias das **rotas** percorridas por cada veículo.



Problema do Roteamento de Veículos - PRV (3/10)

- Para resolver o PRV é preciso conhecer de antemão
 - A distância (custo) entre o depósito e cada cliente
 - A distância entre dois clientes quaisquer.
- **Exemplos:** *distância entre A e B, distância entre B e A (nem sempre é igual!), distância entre E e H, distância entre I e J etc.*



Problema do Roteamento de Veículos - PRV (4/10)

- Por que o PRV é importante ?
 - Ele possui diversas aplicações práticas para o comércio, indústria e governo:
 - Serviços de entrega bancária
 - Entrega de mercadorias
 - Coleta de lixo
 - Definição de rotas de ônibus municipais
 - Definição de rotas mais rápidas entre 2 localidades
 - ...

Problema do Roteamento de Veículos - PRV (5/10)

- Por que o PRV é importante ?
 - A resolução de um PRV proporciona **economia** de gastos e **atendimento satisfatório** aos clientes.
 - Economia de combustível (veículos percorrerão rotas mais curtas)
 - Economia de horas trabalhadas.
 - Serviços realizados sem atraso.
 - Entregas em tempo inferior ao concorrente.
 - Viagens mais rápidas
 - Etc.

Problema do Roteamento de Veículos - PRV (6/10)

- Entretanto... o PRV é um probl. de otimização combinatória de **difícil solução**.
 - Apesar das vantagens que a solução de um PRV proporciona para uma empresa, tentar encontrar sua **solução exata** quase **nunca é viável** computacionalmente.
 - Suponha que a empresa possua apenas **1 veículo** e tenha que visitar **4 clientes** diferentes, iniciando e encerrando sua viagem na garagem da empresa.

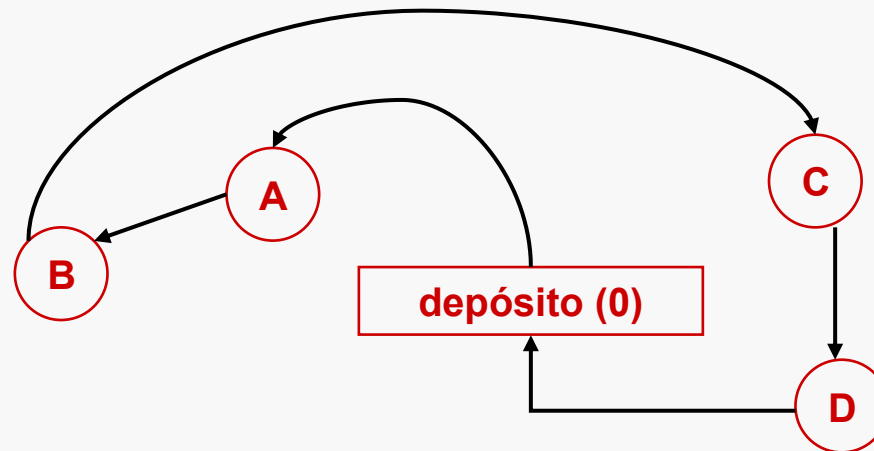
• $k = 1$ veículo

$n = 4$ clientes: A, B, C e D



Problema do Roteamento de Veículos - PRV (7/10)

- Exemplo – Por que o PRV tem custo computacional tão alto ?
 - Um exemplo de rota seria: “*saia da garagem e vá para o cliente A. Daí vá para o cliente B e desse vá para C. Depois vá para D e então volte para a garagem*”.
 - Rota: 0-ABCD-0



Problema do Roteamento de Veículos - PRV (8/10)

- Exemplo – Por que o PRV tem custo computacional tão alto ?

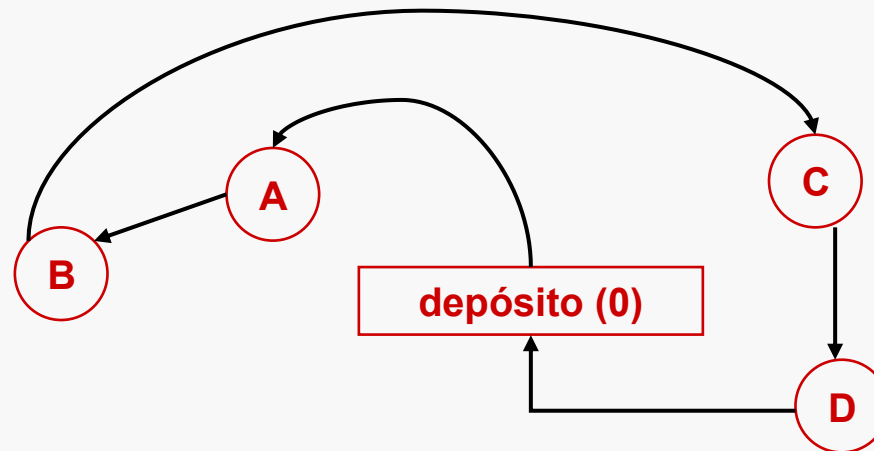
No entanto, existem outras 23 possibilidades de rotas além de 0-ABCD-0:

0-ABDC-0
0-ADCB-0
0-BCDA-0
0-CADB-0
0-CDBA-0
0-DBCA-0

0-ACBD-0
0-BACD-0
0-BDAC-0
0-CBAD-0
0-DABC-0
0-DCAB-0

0-ACDB-0
0-BADC-0
0-BDCA-0
0-CBDA-0
0-DACB-0
0-DCBA-0

0-ADBC-0
0-BCAD-0
0-CABD-0
0-CDAB-0
0-DBAC-0



Problema do Roteamento de Veículos - PRV (9/10)

- Exemplo – Por que o PRV tem custo computacional tão alto ?

Para ter certeza de qual é a melhor... Só medindo e comparando todas

0-ABDC-0
0-ADCB-0
0-BCDA-0
0-CADB-0
0-CDBA-0
0-DBCA-0

0-ACBD-0
0-BACD-0
0-BDAC-0
0-CBAD-0
0-DABC-0
0-DCAB-0

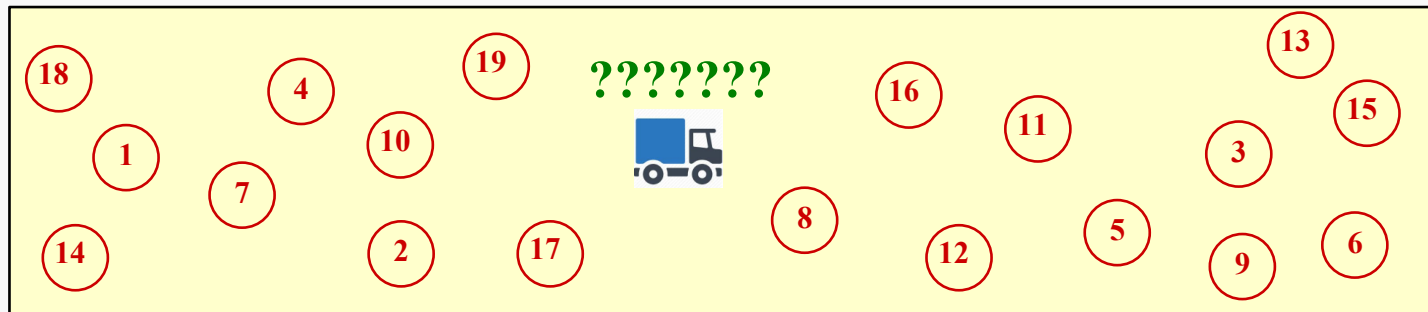
0-ACDB-0
0-BADC-0
0-BDCA-0
0-CBDA-0
0-DACB-0
0-DCBA-0

0-ADBC-0
0-BCAD-0
0-CABD-0
0-CDAB-0
0-DBAC-0



Problema do Roteamento de Veículos - PRV (10/10)

- Exemplo – Por que o PRV tem custo computacional tão alto ?
 - O número R de rotas possíveis para $k = 1$ veículo e n cidades é dado por: $R = n!$
 - Se aumentamos um pouco n , o R torna-se imenso !!! Por exemplo, para $n = 19$
 - $R = 19! = 121.645.100.408.832.000$ rotas diferentes
 - Um computador que faz 1bi de adições/segundo levaria 72 anos para computar...
 - É aí que entra em cena o **algoritmo genético**.



Heurística e Metaheurística (1/3)

- AG é um Metaheurística
 - Mas antes de metaheurística ... vamos definir heurística :
 - Definição na Língua Portuguesa (Dicionário O GLOBO):
 - Heurística. “*arte de inventar ou determinar um método que leva a descobrir a verdade*”.
 - Na Ciência da Computação (Wikipedia):
 - Heurística. “*técnica para resolver um problema mais rapidamente quando um método clássico é muito lento ou para encontrar uma solução aproximada quando os métodos clássicos não conseguem encontrar a solução exata*”.
 - A heurística **troca exatidão por velocidade**. Não garante a descoberta da solução exata de um problema. Mas quase sempre resulta numa boa solução.

Heurística e Metaheurística (2/3)

- Nesse curso, já apresentamos métodos heurísticos !!!
 - **CART** – utiliza o Índice de Gini para gerar uma boa AD
 - Mas não é necessariamente a melhor...
 - Dada uma base de treino, seria preciso gerar todas as ADs possíveis e verificar as acurácias de treino p/ identificar a melhor.
 - **k-Médias** – chuta centro de clusters iniciais e os movimenta por um número de iterações, esperando que k bons grupos sejam definidos no fim do processo.
 - Mas não garante que será produzido o resultado com menor soma do erro quadrado intra-cluster (inertia – [ver aula sobre clustering](#))
 - Seria preciso testar todas as composições possíveis para os k grupos p/ saber qual delas possui a menor inertia.

Heurística e Metaheurística (3/3)

- E o que é Metaheurística ?
 - Trata-se de um conjunto de **estratégias genéricas** que servem como um *template* para levar algoritmos heurísticos ao sucesso.
 - Basicamente: uma **estrutura geral** ou um *framework* pré-definido onde você encaixa um algoritmo heurístico
 - Por ser uma estratégia genérica, pode ser usada para resolver um grande número de problemas diferentes.
 - Se você conseguir **modelar o seu problema na estrutura exigida** pela metaheurística, poderá utilizá-la.

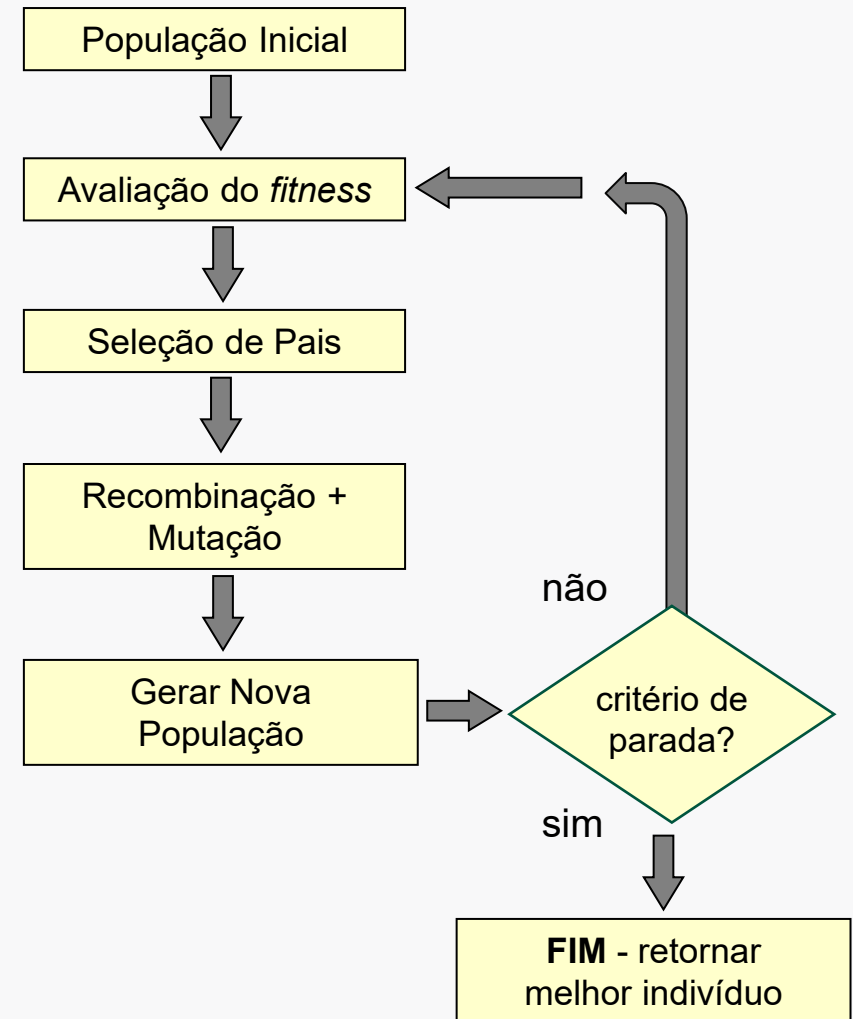
Metaheurística do AG (1/2)

- **Algoritmos Genéticos (AGs)**

- Metaheurística que simula processos biológicos para resolver problemas difíceis.
- A inspiração vem da teoria de **Darwin** de **seleção natural** das espécies.
 - Na **natureza** os **indivíduos competem** entre si por recursos como comida, água e refúgio.
 - Entre os animais de uma mesma espécie, aqueles **menos aptos** para a competição tem **menor chance de sobrevivência**.
 - Desta forma, seus genes têm menor chance de serem propagados ao longo de sucessivas gerações.
 - Por outro lado, os **mais aptos sobrevivem**.
 - Com isto, seus genes têm maior chance de serem propagados ao longo de diversas gerações.

Metaheurística do AG (2/2)

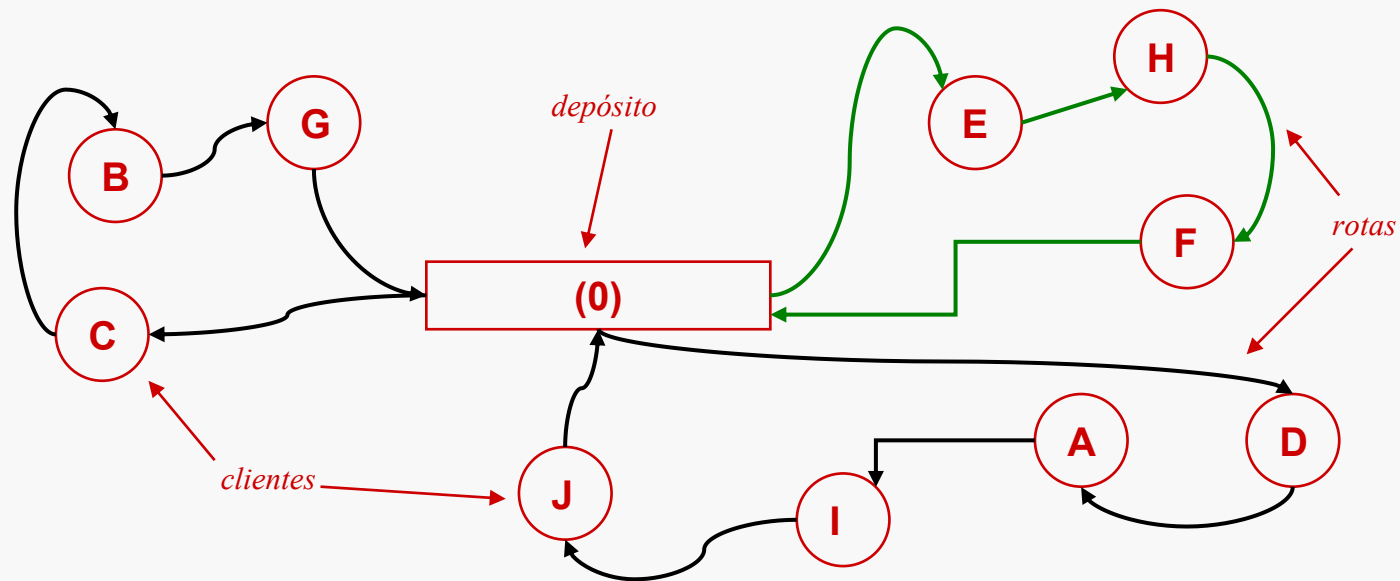
- **AGs** – simulam o processo de seleção natural para resolver problemas difíceis
- Cria-se uma **população de soluções (indivíduos ou cromossomos)** para um dado problema.
- Esta população de soluções evolui ao longo de diversas iterações (**gerações**), de acordo com os princípios da seleção natural:
 - As soluções de maior qualidade (mais **aptas**) sobrevivem e as de menor qualidade morrem. Elas transmitem seus genes (realizam **cruzamentos**) ao longo das diversas iterações.
 - Ao final do processo, o **indivíduo mais saudável** (melhor solução) é selecionado para servir como resposta do problema difícil.



AG em ação (1/17)

• Descrição do Problema

- Suponha uma empresa que possua uma frota de k veículos com capacidade idêntica c e que possua n clientes.
- Desejaremos obter a rota que atenda a maior demanda possível, considerando como critérios secundários o menor número de veículos e a rota mais curta



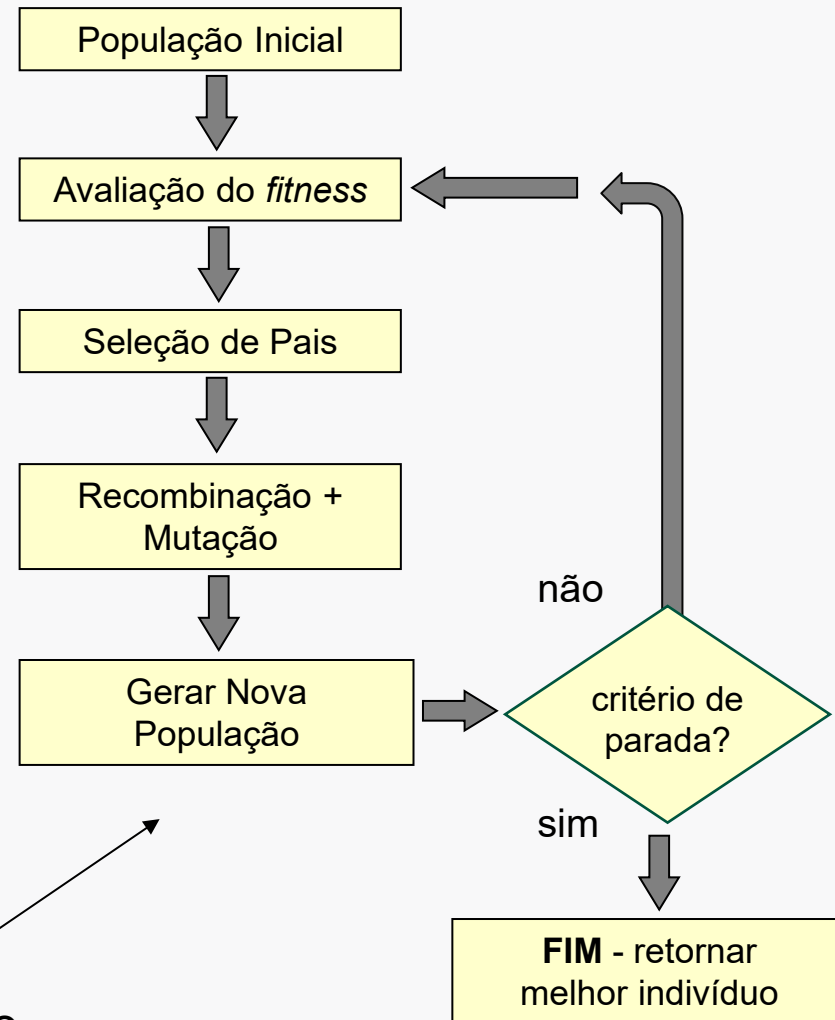
* O AG que será apresentado foi proposto em Tavares et al. (2002)

AG em ação (2/17)

- Etapas para resolver um problema com AG

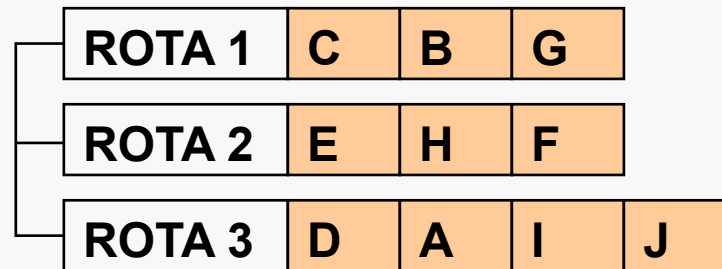
- (0) Definição do formato do cromossomo.
- (1) Geração da população inicial.
- (2) Avaliação da aptidão dos cromossomos.
- (3) Seleção dos pais (reprodutores)
- (4) Recombinação e Mutação.
- (5) Definição da nova população (nova geração)
- (6) Definição Critério de Parada.

- Para usar um AG, você tem que:
 - Encaixar o seu problema nesse modelo
 - E definir os 6 itens acima.



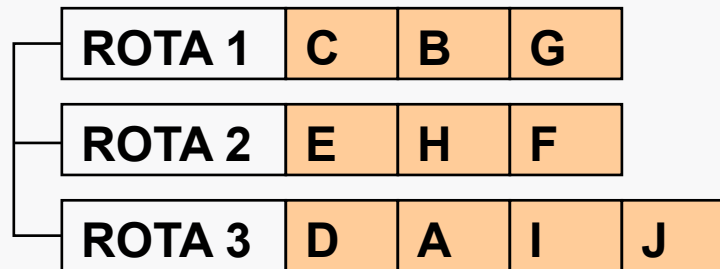
AG em ação (3/17)

- **PASSO PRELIMINAR**: definir a maneira de representar os cromossomos
 - Os cromossomos formarão a população de soluções candidatas.
 - Sendo assim, neste estudo de caso, cada cromossomo deve informar:
 - A divisão das demandas entre os veículos da frota.
 - A ordem em que os clientes serão visitados em cada rota.
- **IMPORTANTE**: um cromossomo **sempre** deve conter uma **solução válida** para o problema (nesse caso, o PRV).



AG em ação (4/17)

- **PASSO 1**: geração da população inicial de cromossomos
 - Nesta etapa, deve-se definir:
 - O tamanho da população (quantidade de cromossomos) .
 - *Pode ser um hiperparâmetro selecionado pelo usuário.*
 - O processo que será utilizado para gerar cada cromossomo da primeira geração.



AG em ação (5/17)

- **PASSO 1: geração da população inicial de cromossomos**
 - Exemplo no PRV:
 - Empresa com frota de $k = 5$ veículos.
 - $n = 10$ clientes.
 - Capacidade máxima de cada veículo = 50.
 - Demandas dos clientes apresentadas na tabela ao lado.

Cliente	Demanda
1	25
2	10
3	11
4	12
5	30
6	15
7	16
8	31
9	22
10	8

AG em ação (6/17)

- **PASSO 1: geração da população inicial de cromossomos**
 - Cada cromossomo recebe uma lista de clientes ordenada de acordo com um sorteio realizado.
 - **Ex.:** cromossomo 1 recebe a lista:
 - 10 – 5 – 2 – 1 – 9 – 8 – 6 – 7 – 4 – 3

Cliente	Demanda
1	25
2	10
3	11
4	12
5	30
6	15
7	16
8	31
9	22
10	8

AG em ação (7/17)

- **PASSO 1: geração da população inicial de cromossomos**
 - As rotas são criadas de acordo com a ordem dos clientes na lista.
 - 10 – 5 – 2 – 1 – 9 – 8 – 6 – 7 – 4 – 3
 - As demandas não podem ultrapassar a demanda máxima de uma rota (= 50).

ROTA 1	10	5	2
ROTA 2	1	9	
ROTA 3	8	6	
ROTA 4	7	4	3

Cliente	Demanda
1	25
2	10
3	11
4	12
5	30
6	15
7	16
8	31
9	22
10	8

- Veja que na solução acima, o 5º veículo da frota não precisou ser utilizado

AG em ação (8/17)

- PASSO 1: geração da população inicial de cromossomos**

- Se o tamanho da população for 100, o processo é repetido 100 vezes...
- ... gerando uma **população inicial** de 100 cromossomos
- ... ou seja, 100 **soluções candidatas**, diferentes uma da outra.

cromossomo 1

ROTA 1	10	5	2
ROTA 2	1	9	
ROTA 3	8	6	
ROTA 4	7	4	3

...

cromossomo 100

ROTA 1	4	6	
ROTA 2	8	7	
ROTA 3	5	2	
ROTA 4	3	10	9
ROTA 5	1		

AG em ação (9/17)

- **PASSO 2: avaliação da aptidão (*fitness*) de cada cromossomo**
 - *Vamos supor* que no nosso problema, o *fitness* de um cromossomo é obtido através do somatório das demandas atendidas.
 - **Não parece ser a melhor escolha**, mas faremos essa suposição inicial e depois vamos melhorar a função
 - Dessa forma, quanto maior o valor do *fitness*, mais saudável é o indivíduo.

Cliente	Demanda
1	25
2	10
3	11
4	12
5	30
6	15
7	16
8	31
9	22
10	8

ROTA 1	4	6	<i>fitness = 27</i>	
ROTA 2	8	7	<i>fitness = 47</i>	
ROTA 3	5	2	<i>fitness = 40</i>	
ROTA 4	3	10	9	<i>fitness = 41</i>
ROTA 5	1		<i>fitness = 25</i>	

$$\text{fitness cromossomo} = 27 + 47 + 40 + 41 + 25 = 180$$

AG em ação (10/17)

- PASSO 2: avaliação da aptidão (*fitness*) de cada cromossomo**

- Entretanto, usando apenas a soma das demandas como *fitness*, pode haver muitos **casos de empate**.

- Para resolver, podemos utilizar a chamada **abordagem lexicográfica**:

- 1º critério:** soma das demandas atendidas (quanto maior, melhor)
- Se houver empate, podemos desempatar pelo **2º critério:** número de veículos usados (quanto menos melhor)
 - No caso ao lado, o cromossomo A empataria com o cromossomo B no 1º critério, mas venceria no 2º.
- Se continuar havendo empate, vamos para o **3º critério:** soma do comprimento das rotas (quanto menor, melhor)

cromossomo A

ROTA 1	10	5	2
ROTA 2	1	9	
ROTA 3	8	6	
ROTA 4	7	4	3

cromossomo B

ROTA 1	4	6	
ROTA 2	8	7	
ROTA 3	5	2	
ROTA 4	3	10	9
ROTA 5	1		

AG em ação (11/17)

- **PASSO 3: seleção dos pais (reprodutores)**

- Pode-se utilizar o **método da roleta** para definir os “pais” que gerarão os indivíduos da próxima geração.
 - **Método da Roleta – pode funcionar assim (dentre outras formas):**
 - O método da roleta atribui uma chance (probabilidade) maior de ser selecionado como pai para os indivíduos com maior fitness.
 - **Por exemplo**, “um cromossomo com bom fitness tem uma chance 5 vezes maior de ser pai quando comparado com um cromossomo com fitness ruim”.
 - Além do método da roleta, existem outros métodos como o **torneio**.
 - Sorteia-se k indivíduos e eles “batalham” para ver quem será selecionado como reprodutor.
 - Basicamente, vence a batalha quem tem o melhor fitness.

AG em ação (12/17)

- PASSO 3: seleção dos pais (reprodutores)

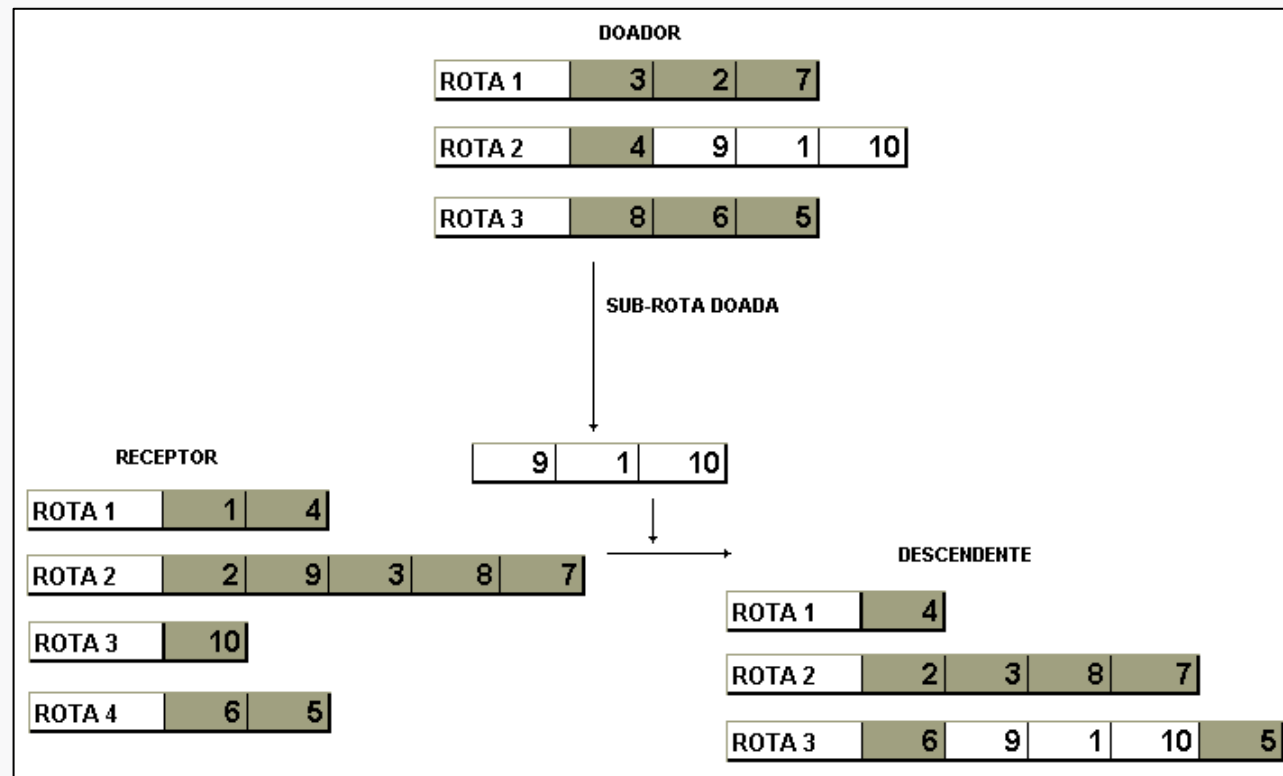
- Se nossa população é de 100 indivíduos, podemos selecionar 100 “casais” de reprodutores, utilizando a roleta ou torneio.
- Ou podemos usar o **elitismo**.
 - Os k melhores indivíduos da geração corrente passarão automaticamente para a próxima
 - Onde $k = 1$ ou 2 ... ou outro valor pequeno
 - Assim em vez de 100 “casais”, serão selecionados $100 - k$.
 - *k também pode ser um hiperparâmetro selecionado pelo usuário*

AG em ação (13/17)

- **PASSO 4(a): recombinação (ou cruzamento – *crossover*)**
 - No passo anterior, os indivíduos que serão os reprodutores foram selecionados.
 - Agora aplica-se o operador genético de ***crossover*** para a que seja possível o “nascimento” de novos indivíduos (metade restante da população).
 - No exemplo do PRV, poderia funcionar assim:
 - Seja um par de cromossomos reprodutores.
 - Um deles é escolhido como “***doador***”.
 - O doador “doa” um fragmento do seu material genético (uma sub-rota) para o outro cromossomo, chamado de “***receptor***”.
 - Este cromossomo “receptor” insere a sub-rota em uma de suas rotas.
 - Assim, nasce o “***descendente***” (indivíduo da próxima geração).

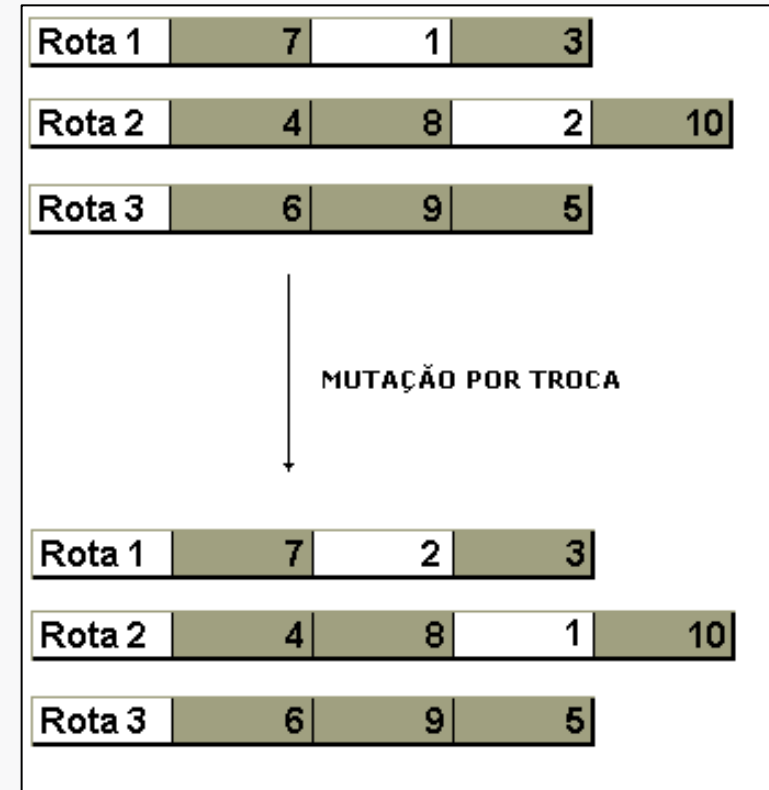
AG em ação (14/17)

- **PASSO 4(a): recombinação (ou cruzamento – *crossover*)**
 - O descendente deve ser ajustado para que não contenha clientes repetidos em outras rotas.



AG em ação (15/17)

- **PASSO 4(b): aplica-se o operador genético de mutação**
 - A mutação é um fenômeno pouco frequente na natureza.
 - É resultante de uma deformação genética, que pode ser passada de um pai para um filho.
 - No AG é usada com o intuito de ajudar o processo a escapar de ótimos locais.
 - No artigo do GA para PRV, escolheu-se apenas um indivíduo por geração para sofrer mutação.
 - Uma sub-rota é selecionada num cromossomo “recém-nascido” e inserida em outro lugar. Este “outro lugar” pode ser outra posição dentro de sua rota original ou uma outra rota.



AG em ação (16/17)

- PASSO 5: Definição da nova geração
 - De uma geração para outra:
 - “mata-se” todos os indivíduos da última geração, exceto os k melhores se usarmos elitismo ...
 - ... E trocamos pelos novos indivíduos gerados pela recombinação.

AG em ação (17/17)

• PASSO 6: Critério de Parada

- O processo evolucionário será executado por **muitas** iterações (**gerações**)
 - Ou seja: os passos 2 a 5 do AG (avaliação do fitness, seleção dos pais, recombinação, mutação e definição da nova geração)
- Com isso, o AG produzirá, progressivamente, gerações de cromossomos (soluções para o problema) cada vez melhores.
- Normalmente, a execução do AG irá terminar quando um indivíduo “saudável o suficiente” surgir ou, mais comumente, **após um número máximo de gerações** ter sido produzido.
 - O **melhor cromossomo** (o de melhor fitness) **da última geração** representará a **solução do problema**.
 - Não sabemos se será a melhor solução do problema, mas provavelmente será uma boa solução.

Resolvendo outros problemas com AGs

- Inúmeros outros problemas de otimização podem ser resolvidos c/ AGs
 - A aplicação do AG não está limitada ao PRV !!! Nessa aula, o PRV foi usado apenas para apresentar o AG de forma prática.
 - Se você tem um problema de otimização, para resolvê-lo com AG terá que **modelar o problema** dentro do **framework do AG**. Isso significa definir:
 - O **formato do cromossomo**
 - O tamanho da população inicial
 - A **função de fitness**
 - O **processo de seleção** de indivíduos para a recombinação
 - Os operadores de **recombinação** e mutação (principalmente o primeiro)
 - O critério de parada
 - AG é uma técnica popular. Muitos trabalhos na literatura poderão servir de base para ajudar em sua modelagem.

k-Médias com AGs (1/5)

- **AGs para melhorar o desempenho do k-Médias** (Maulik e Bandyopadhyay, 2000)
- **Representação do Cromossomo**
 - Cada cromossomo é representado por uma sequência de números reais que representa os k centros dos clusters.
 - **Ex.:** $k = 4$ (queremos encontrar 4 clusters) e BD possui 3 dimensões ($d = 3$).
 - O comprimento do cromossomo será dado por $d * k = 3 * 4 = 12$.
 - Exemplo de uma representação de cromossomo para este caso:

$[(1.0, 2.0, 1.5), (1.9, 3.8, 2.0), (1.8, 2.5, 2.9), (2.5, 4.0, 3.7)]$

k-Médias com AGs (2/5)

- **Recombinação**

- É utilizada a forma chamada *Single Point Crossover*.
- Um ponto chamado *crossover point* é escolhido. Então, para gerar o filho fazemos um seguinte: no primeiro pai copiamos o conteúdo do cromossomo até o *crossover point* e no segundo pai copiamos do *crossover point* em diante. (*no exemplo abaixo, considerando $k = 3$*)

- PAI 1: [(20.53, 70.28), (14.50, 60.82), (30.13, 45.45)]

- PAI 2: [(15.00, 60.05), (13.15, 45.90), (28.76, 47.20)]

|--- **Crossover Point**

- FILHO: [(20.53, 70.28), (14.50, 60.82), (28.76, 47.20)]

k-Médias com AGs (3/5)

• Fitness

- Os clusters são formados de acordo com os centros representados no cromossomo em consideração.
- P/ cada cromossomo, definimos os pontos $p_1, p_2, \dots, p_n$ de cada cluster $C_1, C_2, \dots, C_k$.
- Depois, novos centros dos clusters $c_1, c_2, \dots, c_k$ são recalculados da mesma forma feita pelo k -Médias
 - O cromossomo é alterado para conter estes novos centros.
 - A qualidade do cluster é medida pela inertia:

$$Inertia = \sum_{i=1}^k \sum_{p \in C_i} dist(p, c_i)^2$$

- A função de fitness foi definida pelos autores como $f = 1 / inertia$.
- A maximização de f leva a minimização da $inertia$.

k-Médias com AGs (4/5)

- **Seleção de Pais**
 - É utilizado o método da roleta.
 - Ao cromossomo é associado um número de cópias ou um peso que é proporcional ao seu fitness na população.
 - Assim sendo os cromossomos com mais “saudáveis” possuem maior probabilidade de serem escolhidos para reprodução.

Indivíduo	1	2	3	4	5	6	7	8	9	10	11
Valor do fitness	2.0	1.8	1.6	1.4	1.2	1.0	0.8	0.6	0.4	0.2	0.0
Probab. de seleção	0.18	0.16	0.15	0.13	0.11	0.09	0.07	0.06	0.03	0.02	0.0

k-Médias com AGs (5/5)

- **População Inicial**

- Os k centros de cluster codificados em cada cromossomo são inicializados para k pontos aleatoriamente escolhidos da BD.
- O processo é repetido para cada um dos m cromossomos da população (m é escolhido pelo usuário).

- **Critério de Parada**

- O algoritmo é executado até um número máximo de iterações. O melhor cromossomo da última geração fornece a solução para o problema de clusterização.
- Foi implementado **elitismo** a cada geração para preservar sempre o melhor cromossomo

LLMs com AGs (1/2)

- **Evolução de prompts de LLMs com AGs** (Guo et al., 2024)
 - Nos dias atuais, LLMs (GPT, Llama, Gemma etc.) vem sendo utilizadas para resolver diversos tipos de problemas.
 - Incluindo problemas de classificação, como análise de sentimentos e classificação de discurso de ódio
 - Nessa abordagem, um desafio é encontrar o *prompt* que faça a LLM **resolver o problema da melhor forma possível**.

“Best Prompt: You are analyzing a social media post. If the text contains hate speech, offenses, aggressions, insults, swear words, or any form of toxicity, respond only with ‘yes’. If it is appropriate and non-toxic, respond ‘no’”.

Fonte: Oliveira et al. (2024)

LLMs com AGs (2/2)

- **Identificando o melhor prompt com AG**

- Exemplo para classificação de textos tóxicos, baseado na ideia de Guo et al. (2024)
 - (i) inicializa a população de prompts, cada um especialmente designado para classificar textos em “tóxico” ou “não tóxico”
 - (ii) Mantém só os k prompts de melhor desempenho, usando a medida de F1 como função de fitness.
 - (iii) Aplica operações para evoluir o prompt gerando novas variações através da recombinação dos elementos dos prompts bem sucedidos.
 - (iv) o processo é repetido por um número de ciclos, até que o melhor prompt do último ciclo é retornado.

Seleção de Atributos com AGs (1/3)

- **Seleção de Atributos para Classificadores**
 - **PROBLEMA:** identificar o subconjunto de atributos mais relevantes para a tarefa de classificação.
 - Ou seja: os atributos que levarão à construção do classificador com melhor desempenho preditivo.
 - **DESAFIO:** existem 2^n possíveis subconjuntos a serem levados em consideração, onde n é o número total de atributos,
 - Dessa forma, a de todos esses subconjuntos é computacionalmente inviável
 - **SOLUÇÃO:** usar AG para explorar o espaço de busca de forma inteligente, obtendo uma “boa solução” (bom subconjunto)

Seleção de Atributos com AGs (2/3)

- **Representação do Cromossomo**

- **Codificação binária:** O indivíduo é um vetor de bits (0 ou 1) com o mesmo tamanho do número total de atributos existentes.
 - **Ex.:** se há 5 atributos no total, o cromossomo **[1, 0, 1, 1, 0]** indica que o modelo será treinado com o 1º, 3º e o 4º atributos, ignorando o 2º e o 5º.

- **Função de Aptidão (Fitness)**

- A partição de treino é dividida em **treino** e **validação**. Para cada indivíduo:
 - **Treina-se** um modelo **com os atributos marcados como 1** em seu cromossomo, usando a partição de treino.
 - Computa-se o **valor do fitness** na partição de **validação**.
 - Pode ser o F1 por exemplo;
 - Muitas vezes penaliza-se pelo número de atributos escolhidos (para favorecer conjuntos menores).
- **Obs.:** essa abordagem de avaliar a qualidade do cromossomo usando o modelo treinado é conhecida como *wrapper*.

Seleção de Atributos com AGs (3/3)

- **Recombinação**

- Os pares de reprodutores selecionados (pais) trocam pedaços de seus vetores binários para gerar novos indivíduos (filhos). Usa-se o ponto de corte (*crossover point*)

- **Exemplo:**

- Pai 1: [1, 0, 1 | 1, 0]

- Pai 2: [0, 1, 0 | 0, 1]

Filho resultante: [1, 0, 1 | 0, 1] (combina partes dos dois pais)

- **Mutação**

- Para evitar que o algoritmo fique preso em mínimos locais e garantir diversidade genética, alguns bits do vetor mudam aleatoriamente com uma probabilidade muito baixa
 - Por exemplo, 1% de chance de um 1 virar 0 ou vice-versa.

- **Critério de Parada**

- O processo se repete por várias gerações. Ao final, o melhor indivíduo encontrado em todo o processo revela qual é o subconjunto ideal de atributos para o classificador.

Referências

- EIBEN, A. E.; SMITH, J. E. **Introduction to Evolutionary Computing**. 2. ed. Heidelberg : Springer-Berlin, 2015.
- GUO et al. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. *In: The Twelfth International Conference on Learning Representations*, 2024, Vienna. **Anais** [...] [s.l.] : ICLR, 2024.
- KRUSE, R.; BORGELT, C; BRAINE, C.; MOSTAGHIM, S.; STEINBRECHER, M. **Computational Intelligence – A Methodological Introduction**. 3. ed. Cham : Springer, 2022.
- MAULIK, U; BANDYOPADHYAY. S. Genetic algorithm-based clustering technique. *In: Pattern Recognition*, v. 33, n. 9, p. 1455 – 1465, 2000.
- NEVES, J. A. **Uma aplicação de algoritmo genético na otimização do emprego da unidade móvel de pistoneio**. 2000. Dissertação (Mestrado em Sistemas de Computação) – UFRN, 2000.
- OBITKO, M. Genetic Algorithms. **obitko.com**, 1998. Disponível em < <https://www.obitko.com/tutorials/genetic-algorithms/index.php> >. Acesso em: 23 abr. 2025.
- OLIVEIRA et al. Toxic Text Classification in Portuguese: Is LLaMA 3.1 8B All You Need?. *In: STIL*, 2024, Belém. **Anais** [...] Porto Alegre : SBC, 2024.
- PEREIRA, F. B.; TAVARES, J.; MACHADO, P.; COSTA, E. GVR: A New Genetic Representation for the Vehicle Routing Problem. *In: Proc. of the 13th Irish Int'l Conf. on Artificial Intelligence and Cognitive Science*, 2002. Limerick. **Anais** [...] Heidelberg : Springer-Verlag, 2002.